

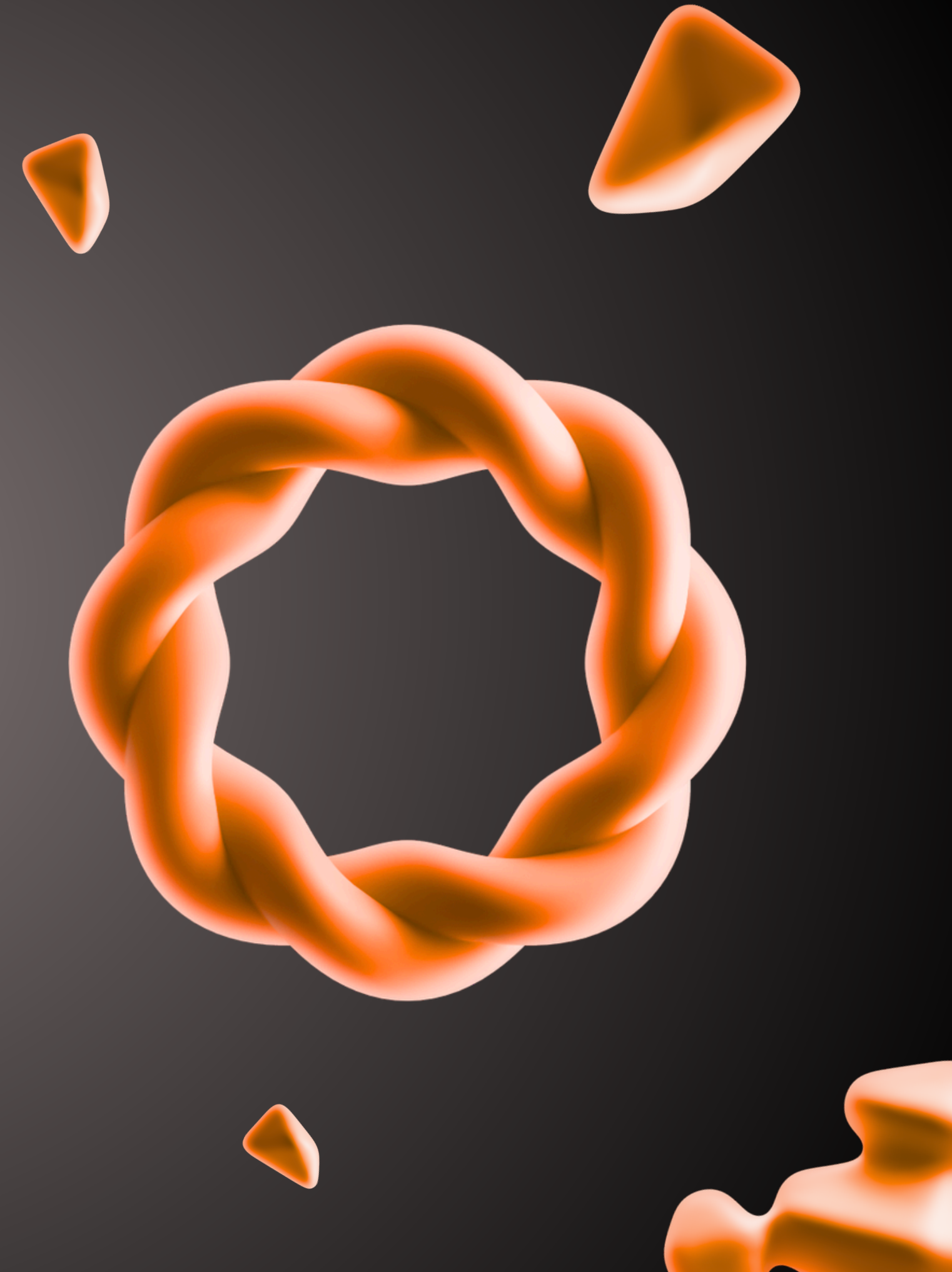
NEXUSTEAM

THWS Project Defense

NexusTeam

Multi-Platform Communication System

Team: Anna Kornet, Sofiia Khyzhnychenko, Vladyslav Zaplitniy, Halil Hakan Karabay



Project Overview & Motivation

SYSTEM CONCEPT

- NexusTeam is a modern multi-platform communication system providing real-time messaging, file sharing, presence, voice calls, and AI image generation.

CORE PROBLEM

- Modern messaging platforms require reliability, security, and scalability — which single-database architectures struggle to deliver efficiently.

ARCHITECTURAL SOLUTION — POLYGLOT PERSISTENCE

- Oracle Database: relational data for structured user accounts and security.
- MongoDB: document-oriented data for chats, messages, folders, and attachments.
- Redis: in-memory caching for high-speed temporary data, sessions, and presence.

REAL-TIME COMMUNICATION

- Persistent WebSocket connections replace inefficient REST polling.

Project Goals & Target Users



INDIVIDUAL & GROUP USERS

- Direct messaging and team collaboration.



DESKTOP USERS (WINDOWS)

- Native WPF client for deep OS integration.



WEB USERS

- Flexible browser-based access without installation.

PRIMARY OBJECTIVES

- Secure user registration and JWT-based authentication.
- Support for private & group chats, real-time exchange, editing, and deletion.
- File sharing, presence tracking, chat folders, voice calls, and translation.
- Clear separation between presentation, business logic, and data access.

Team Organization & Responsibility Distribution

SK Sofiia Khyzhnychenko

Requirements Management, Architecture,
Documentation

AK Anna Kornet

User Interface

VZ Vladyslav Zaplitniy

Backend and Client Implementation

HHK Halil Hakan Karabay

Testing

Full Technology Stack

BACKEND

- ASP.NET Core 8 (C#) — REST API and application logic.

REALTIME COMMUNICATION

- WebSockets for bidirectional event delivery.

DATABASES

- Oracle Database: structured user & account data.
- MongoDB: document-oriented chats, messages, and attachments.
- Redis: in-memory caching, sessions, and presence state.

CLIENTS & INFRASTRUCTURE

WPF (.NET 8)

HTML/CSS/JS

Docker

BCrypt

JWT

Serilog

LiteDB

Requirements Management & System Scope

STAKEHOLDERS

- End Users
- Development Team
- Academic Supervisors (THWS)
- System Administrators

5 CORE AREAS OF SYSTEM SCOPE

1

User Management
Registration, Auth,
Profiles, Search)

2

Communication &
Chat Management
Direct/Group,
Folders)

3

Realtime
Functionality
WebSockets,
Presence, Typing
Indicators)

4

Client Applications
WPF Desktop & Web
Clients)

5

Infrastructure &
Security Docker,
JWT, Rate Limiting,
BCrypt)

Functional Requirements: Core Messaging & User Flow

FR-01 – FR-05

User Auth & Profiles

Registration, Login, JWT session management, profile management, global user search.

FR-06 – FR-08

Chat Management

Direct and group chat creation, participant control, user chat folders.

FR-09 – FR-13

Messaging

Send, edit, delete messages; real-time delivery over WebSockets; read/delivered statuses.

FR-09–FR-13

Messaging

Message replies (threading/quotes), message forwarding with original author attribution, and "Saved Messages" personal cloud chat

FR-14–FR-15

Attachments

Document sharing with format-specific in-app previews

Real-Time & Extended Functional Capabilities

FR-16 – FR-19

Realtime Communication

Persistent WebSockets, live typing indicators, presence (Online/Offline), chat state events.

FR-20 – FR-21

Voice Calls

Start, accept, reject, and end voice call signaling.

FR-22 – FR-24

Supporting Services

User preferences storage, AI image generation, in-chat text translation.

FR-20–FR-21

Voice/Video Calls

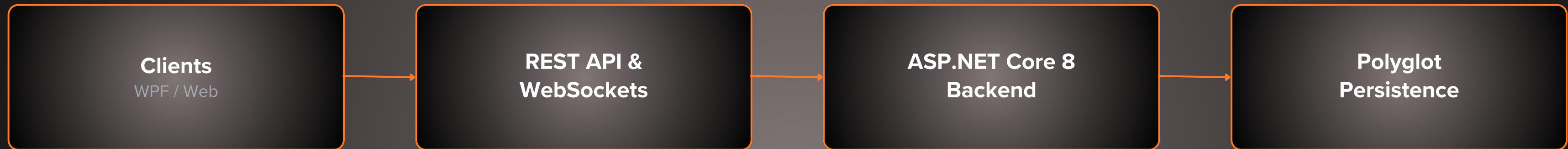
P2P / TURN-assisted audio call session management

FR-22–FR-24

Supporting Services

Custom chat themes and personal wallpaper customization

Client-Server Architecture Overview



SECURITY BOUNDARY

- Clients NEVER access databases directly; all operations are mediated by the backend.

SINGLE SOURCE OF TRUTH

- Business logic, validation, authentication, and permission rules remain centralized on the server.

MULTI-CLIENT SUPPORT

- Desktop and Web clients consume the exact same backend endpoints and WebSocket contracts.

Specialized Database Architecture

1

Oracle Database

Stores structured user/account data, guaranteeing ACID compliance and credential uniqueness.

2

MongoDB

Document database storing chats, message history, attachment metadata, folders, preferences, and generated images.

3

Redis

High-speed in-memory store handling active sessions, user status, typing events, and call signaling state.

ASP.NET Core Server Internal Architecture

Controllers Layer



8 REST controllers (Auth, Users, Chats, Messages, Attachments, Preferences, ChatFolders, GeneratedImages).

Service Layer



Encapsulates business logic (AuthService, MessageService, ChatService, CallService, etc.).

Repository Layer



Data access abstraction via IUserRepository (Oracle) and MongoDB repositories.

Dependency Injection & Middleware



Loose coupling via IoC container; custom middleware for security headers, JWT auth, logging, and error handling.

Real-Time WebSocket Communication Architecture

ENDPOINT & CONNECTION LIFECYCLE

- Routed via /ws; connection is established and authenticated via JWT within a strict timeout window.

WEBSOCKETCONNECTIONMANAGER

- Manages active socket sessions and maps connected users to their active channels.

MESSAGE ENVELOPE PROTOCOL

- All events use WebSocketMessageEnvelope with explicit event types and serialized JSON payloads.

SUPPORTED EVENTS

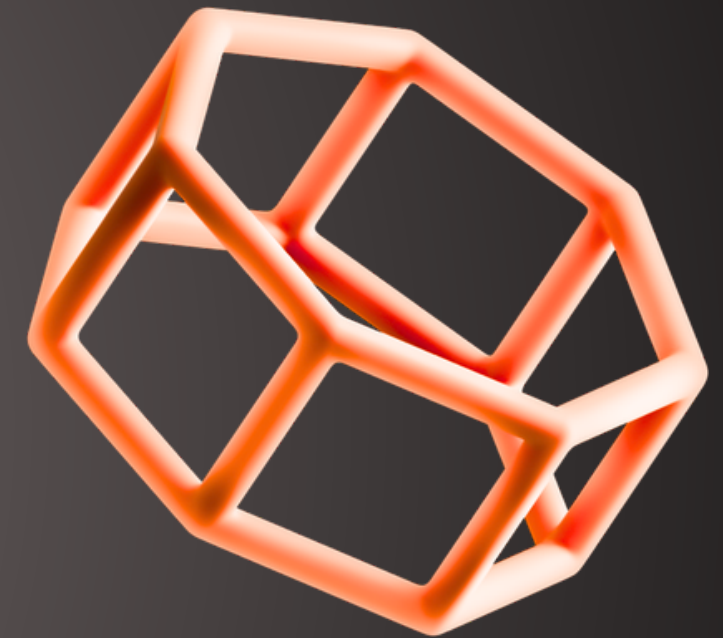
New messages

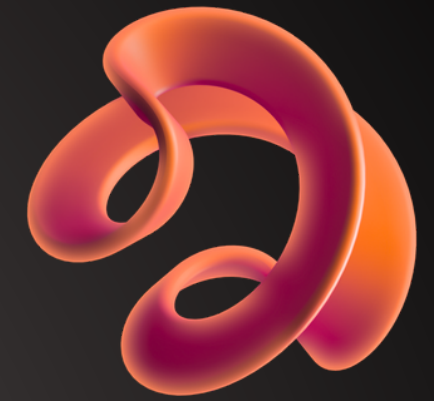
Typing indicators

Read / delivered updates

Presence changes

Call signaling





Shared Contracts & Communication Protocol

PURPOSE

- Single source of definitions shared between Server, WPF Client, and Web Client to eliminate contract drift.

DTOs & MODELS

SavedMessagesDto
MessageAttachmentDto
UserPreferenceDto
ChatFolderDto
UserDto
ChatDto
MessageDto

WEBSOCKET CONTRACTS

UserStatusChangedContract
ChatMessageContract
TypingIndicatorContract
MessageReadContract
CallContracts

ENUMS & UTILITIES

Custom exception classes
Serialization helpers
UserStatus
ChatType
MessageStatus

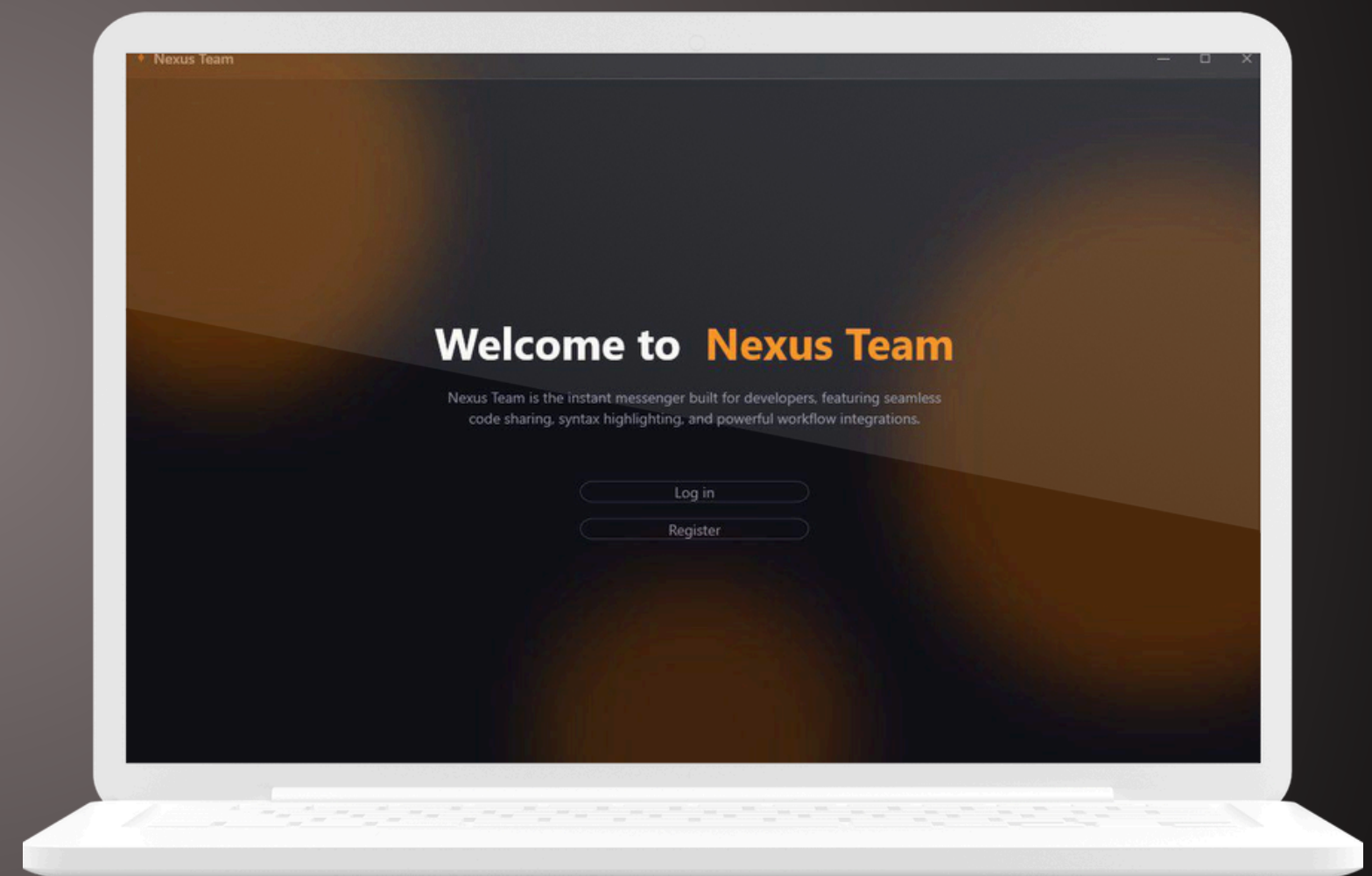
Desktop (WPF) and Web Client Architecture

WPF CLIENT (MVVM)

- Views: XAML-based screens (ChatView, LoginView, CallWindow, GeneratorView, etc.).
- ViewModels: Data binding and command handling (ChatViewModel, CallViewModel, LoginViewModel).
- Client Services: MessagingService, AuthenticationService, OfflineMessageQueue, CredentialStorageService (LiteDB).
- CallOverlayView / CallViewModel, DocumentPreviewService, WallpaperSelectorView

WEB CLIENT

- Lightweight HTML5, CSS3, ES6+ JS client served via Nginx in Docker, communicating with the exact same REST/WS backend.
- WebRTC API



Authentication, Authorization, and Data Protection

NFR-01

Password Hashing

Passwords hashed using BCrypt (work factor 11).

NFR-02 / NFR-03

Token Authentication

JWT Access Tokens for API operations + Refresh Tokens for session persistence.

NFR-04

Authorization

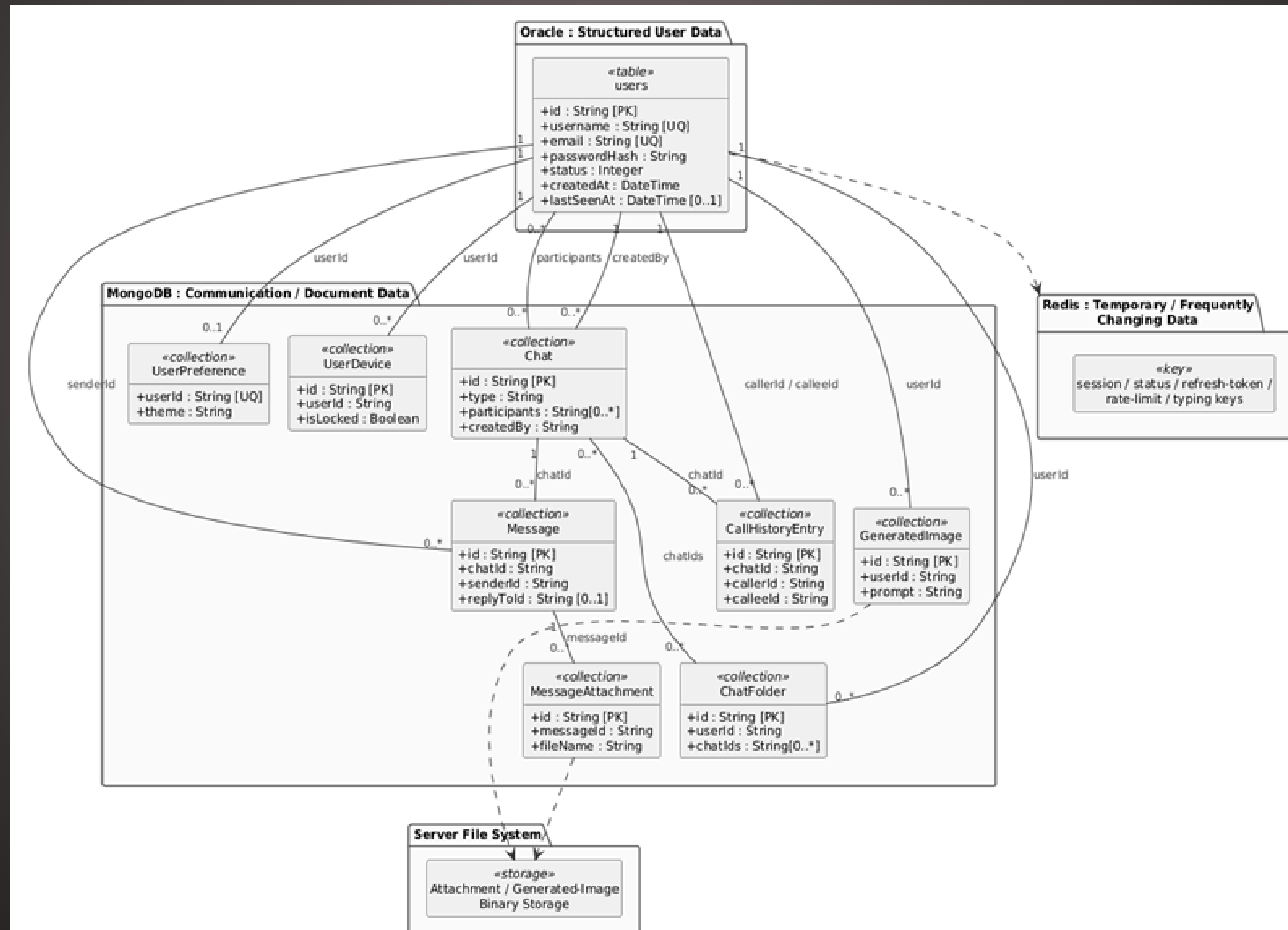
Strict resource ownership checks — users edit/delete only their own messages.

NFR-05 – NFR-07

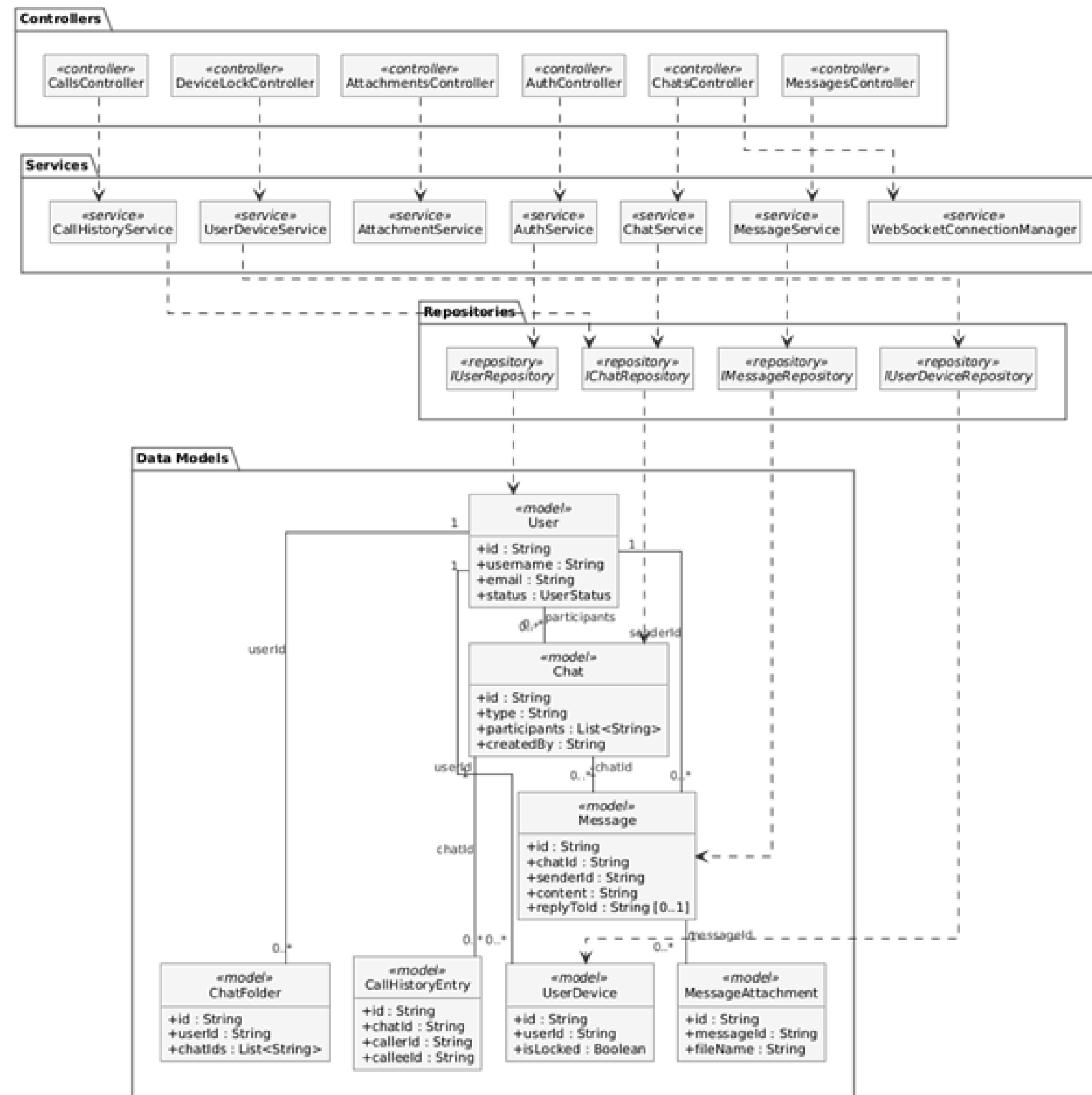
Infrastructure Security

Rate limiting against brute-force/DDoS, authenticated WebSockets, CORS, security headers.

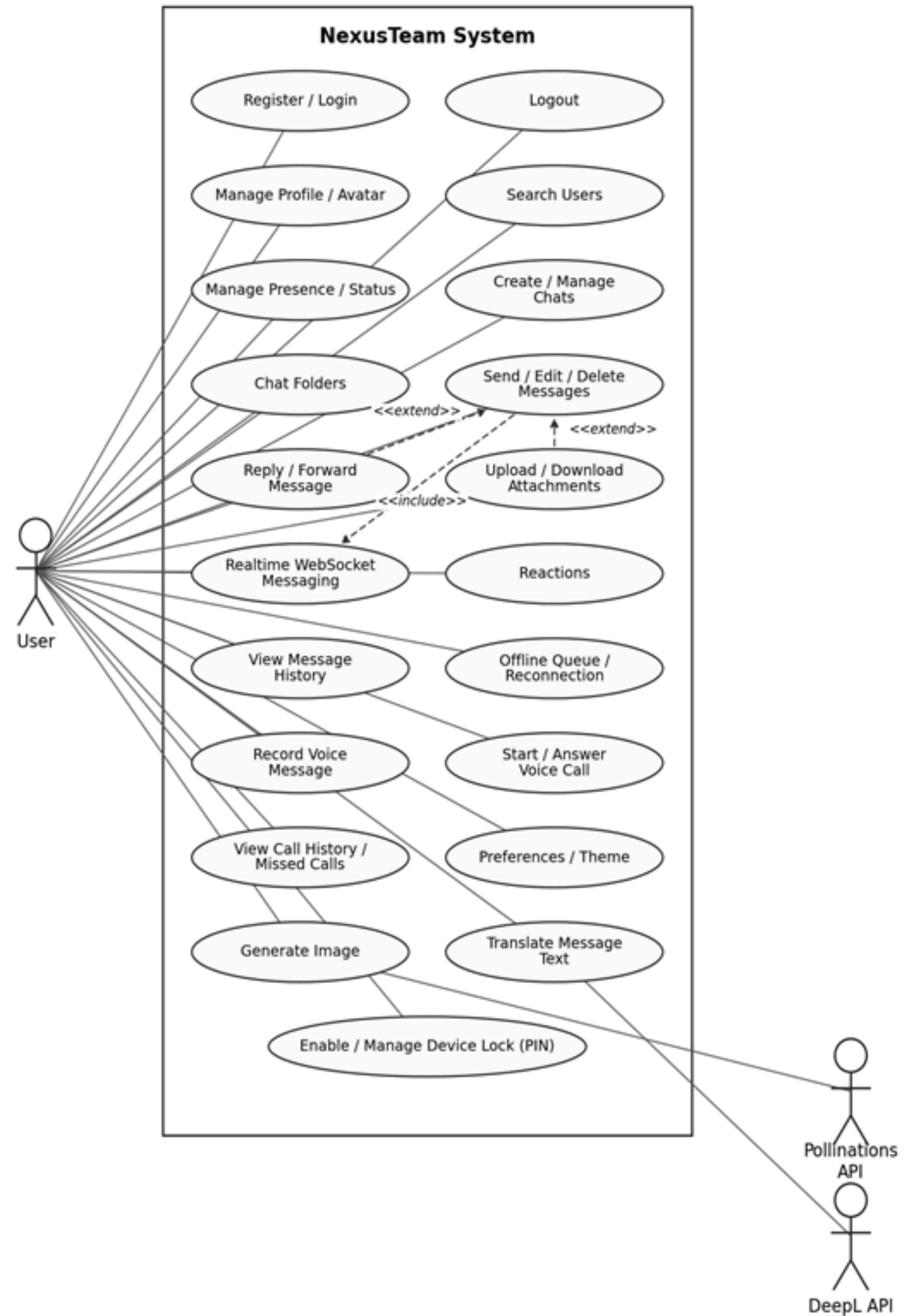
Polyglot Database Schema (Oracle, MongoDB, Redis)



System Class Diagram & Service Interaction



User Interaction & System Use Cases



Comprehensive Testing Approach

TESTING LEVELS

Functional

API

WebSocket

Security

UI

Integration

End-to-End (E2E)

AUTH-01 – AUTH-10

Authentication Valid/duplicate registration, invalid credentials, token expiration.

MSG-01 – MSG-10

Messaging Send, edit, delete ownership, emoji support, real-time receipt.

WS-01 – WS-10

WebSockets Authentication timeout, reconnection recovery, presence broadcasts.

SEC-01 – SEC-10

Security Unauthorized resource access rejection, rate limiting enforcement.

Docker Deployment, Conclusion, and Future Work

CONTAINERIZED DEPLOYMENT

- Docker Compose manages Server, Web Client (Nginx), Oracle, MongoDB (Sharded), Redis, and NexusTeam.DbSeeder.

CONCLUSION

- Successfully delivered a fully functional, modular, multi-platform real-time messaging system meeting all academic requirements.



**THANK YOU FOR YOUR
ATTENTION!**